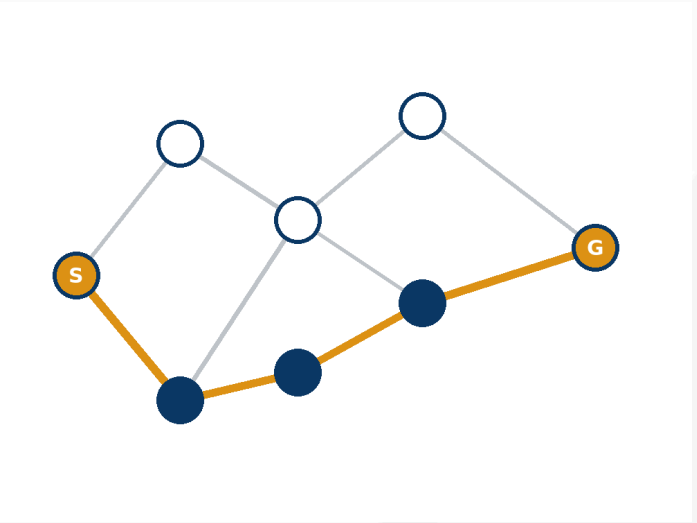


UD02: Modelos de IA y resolución de problemas

Modelos de Inteligencia Artificial

version: 2026-11-03

IES EDUARDO PRIMO MARQUÉS

¿Qué veremos?

1. Sistema de resolución de problemas y búsqueda
2. Clasificación de modelos de IA
3. Automatización de tareas
4. Razonamiento impreciso: lógica difusa
5. Sistemas basados en reglas
6. Adecuación del modelo, y Robocode como caso completo

d.martinezpena@edu.gva.es

RA2 y sus criterios de evaluación

RA2: implementa sistemas de resolución de problemas con modelos de IA.

CE	Criterio
a	Requisitos de un SRP
b, c	Clasificación de modelos de IA y automatización de tareas
d, e	Razonamiento impreciso y sistemas basados en reglas
f	Adecuación del modelo

Hilo conductor de la unidad

Antes de programar IA hay que saber formalizar un problema y elegir bien el modelo.

Modelar → clasificar → automatizar → razonar (difuso y por reglas) → decidir → implementar.

El cierre de la unidad, Robocode, recorre las seis fases con un bot de combate real.

d.martinezpena@edu.gva.es

1. Sistema de resolución de problemas (RA2-a)

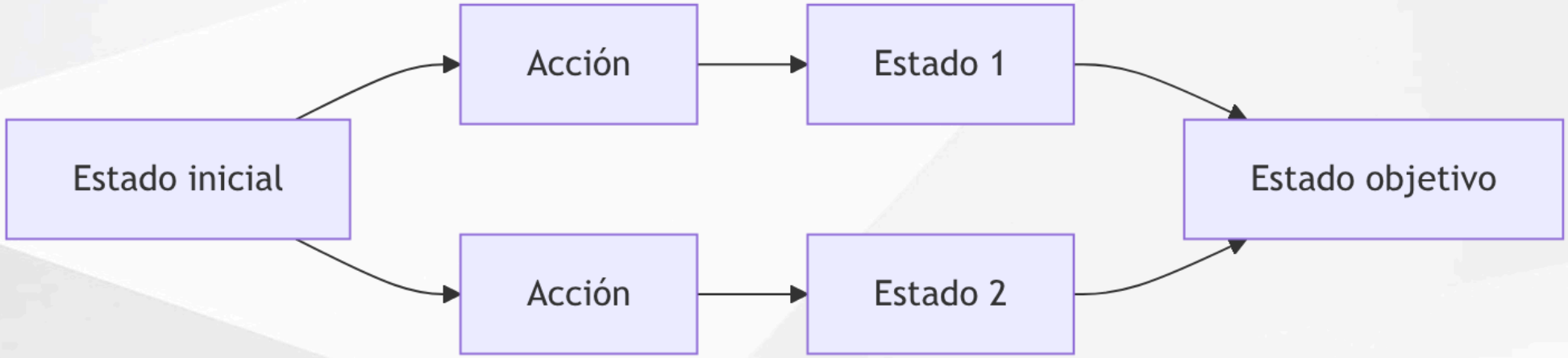
d.martinezpena@edu.gva.es

Cinco requisitos de un SRP

1. **Representación**: elegir estructura y modelo fieles al problema.
2. **Razonamiento y decisión**: lógica, aprendizaje o búsqueda heurística.
3. **Aprendizaje y adaptabilidad**: mejorar con la experiencia.
4. **Eficiencia computacional**: respuestas rápidas y escalables.
5. **Interacción con las personas usuarias**: interfaz comprensible.

d.martinezpena@edu.gva.es

El espacio de estados



Un problema bien planteado responde a: ¿estado inicial?, ¿acciones aplicables?, ¿modelo de transición?, ¿test de objetivo?, ¿coste de camino?

Jarras 8-5-3

Medir una cantidad exacta con jarras sin marcas: llenar, vaciar o verter entre jarras.

Estado `[jarra8, jarra5, jarra3]`: de `[8,0,0]` a `[4,4,0]` en 7 pasos.

Interesa el camino más corto (menos trasvases) → encaja con **BFS**.

Misioneros y caníbales

3 misioneros y 3 caníbales cruzan un río en una barca de 1-2 plazas, sin que los caníbales superen nunca a los misioneros en ninguna orilla.

Estado `⟨m,c,barca⟩`: de `⟨3,3,1⟩` a `⟨0,0,0⟩`.

Hay estados prohibidos que podar → backtracking (DFS), o BFS si importa el número de cruces.

8-reinas

Colocar 8 reinas en un tablero 8×8 sin que ninguna ataque a otra (fila, columna, diagonal).

Representación: una columna por reina → 40.320 permutaciones, **92 soluciones**.

No hay camino más corto que buscar: es satisfacción de restricciones (CSP) → **backtracking**.

15-puzzle

Mover fichas en una matriz 4×4 hasta el orden objetivo.

≈$10{,}46 \cdot 10^{12}$ estados: BFS y DFS son **inviables** por tamaño.

Aquí la heurística no es opcional: A* con Manhattan distance es la única opción realista.

¿Qué algoritmo para qué problema?

Problema	Qué busca	Algoritmo
Jarras	Camino más corto	**BFS**
Misioneros y caníbales	Camino sin estados prohibidos	**BFS / DFS con poda**
8-reinas	Una solución válida (CSP)	**Backtracking (DFS)**
15-puzzle	Camino óptimo en espacio enorme	**A* con heurística**

La explosión combinatoria

El número de estados crece **exponencialmente** con el tamaño del problema (factor de ramificación `b` elevado a profundidad `d`).

Un espacio real (rutas de reparto, planificación) es enorme: hace falta una estrategia de búsqueda y, cuando se puede, una heurística.

BFS, DFS y A*

Criterio	BFS	DFS	A*
Estructura	Cola (FIFO)	Pila (LIFO)	Cola de prioridad `f=g+h`
Completa	Sí	No	Sí
Óptima	Sí (coste unitario)	No	Sí (heurística admisible)
Cuándo usar	Camino más corto en pasos	Exploración exhaustiva	Ruta óptima con costes variados

BFS: cómo funciona

Explora el espacio de estados **por niveles**, nivel a nivel completo, usando una cola FIFO.

Pros: encuentra siempre el camino más corto en número de pasos; es completa.
Contras: guarda en memoria todos los estados del nivel actual — puede agotar la memoria en espacios grandes.

DFS y backtracking: cómo funciona

Baja hasta el fondo de una rama antes de probar otra, usando una pila (o recursión).

Pros: usa muy poca memoria (solo el camino actual); el **backtracking** descarta de inmediato una rama inválida y retrocede, sin explorarla entera.
Contras: no garantiza la solución óptima; puede no terminar en espacios infinitos sin control.

A*: cómo funciona

Combina el coste ya recorrido `g(n)` con una **heurística** `h(n)` que estima lo que falta (`f(n) = g(n) + h(n)`) y expande siempre el estado de menor `f`.

Pros: completo y óptimo con una heurística admisible; evita explorar ramas que se alejan del objetivo.
Contras: necesita diseñar una buena heurística; sin ella, degenera en un BFS más costoso.

2. Clasificación de modelos de IA (RA2-b)

d.martinezpena@edu.gva.es

Por paradigma de aprendizaje

	Supervisado	No supervisado	Refuerzo
Datos	Etiquetados	Sin etiquetas	Recompensas
Objetivo	Predecir la salida	Descubrir patrones	Maximizar recompensa
Tareas	Clasificación, regresión	Clustering, reducción de dim.	Control, juegos, robótica
Ejemplos	Spam, precio de coche	Segmentar clientes	AlphaGo, robots
Algoritmos	Regresión logística, SVM, árboles	k-means, DBSCAN, PCA	(fuera del stack del curso)

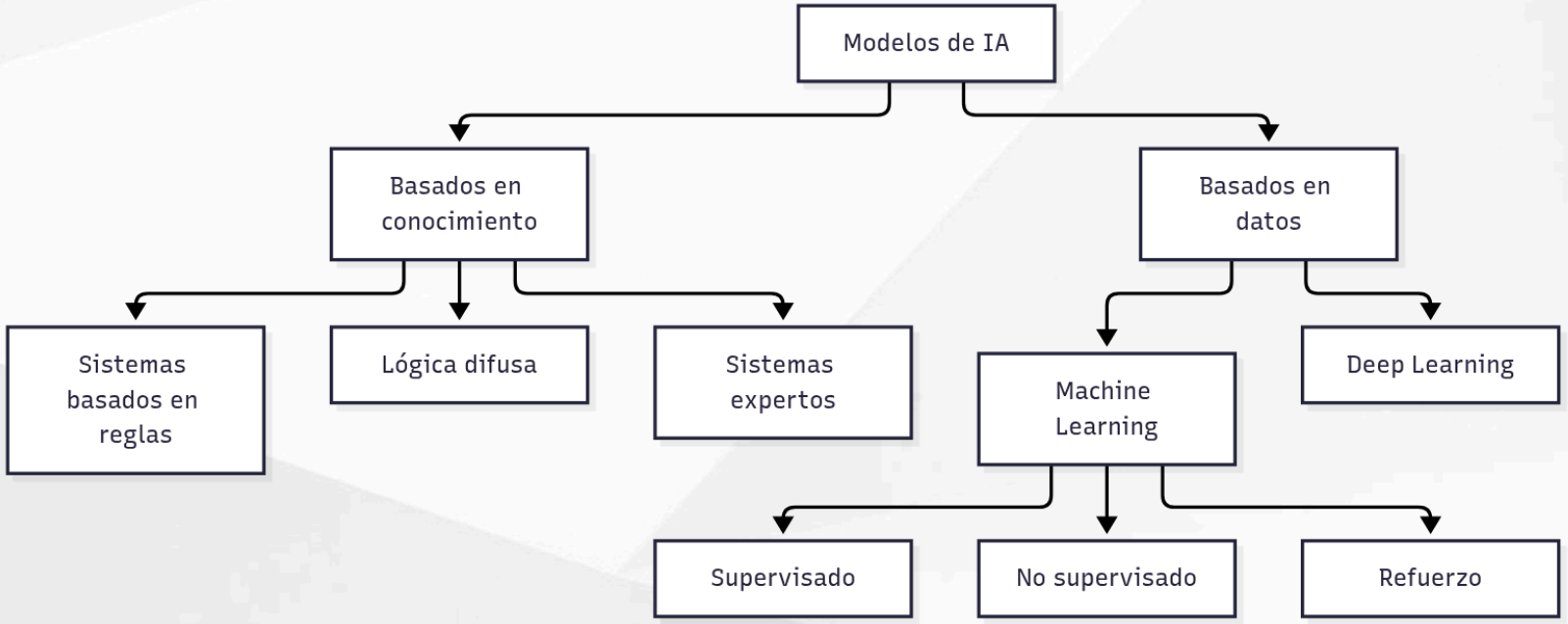
Todo ML es IA, pero no toda IA es ML: reglas y lógica difusa son IA sin aprender de datos.

Por tipo de análisis y por base

Análisis	Pregunta	Ejemplo
Descriptivo	¿Qué pasó?	Informe de ventas
Predictivo	¿Qué pasará?	Previsión de demanda
Prescriptivo	¿Qué hacer?	Precio óptimo a fijar

Basados en **conocimiento** (reglas explícitas, interpretable) frente a basados en **datos** (ML/DL, flexible pero caja negra).

Mapa de los modelos de IA



3. Automatización de tareas (RA2-c)

d.martinezpena@edu.gva.es

RPA frente a IA

	RPA	IA
Qué hace	**Hace**: replica tareas en la interfaz	**Piensa**: reconoce patrones y decide
Base	Reglas predefinidas	Datos y modelos
Se adapta	No	Sí, con la experiencia
Ejemplo	Copiar datos de un email a un ERP	Clasificar si un email es urgente

RPA no es IA: se complementan — la IA decide, el RPA ejecuta. IA + BPM + RPA = automatización inteligente.

Agentes software

Tipo de agente	Cómo decide	Ejemplo
Reflejo simple	Reglas `si... entonces...` directas	Termostato
Reflejo con modelo	Mantiene un estado interno del mundo	Robot aspiradora que "recuerda" la habitación
Basado en objetivos	Busca y planifica para alcanzar una meta	Navegador que elige una ruta
Basado en utilidad	Maximiza una utilidad (coste, tiempo, riesgo)	Ruta que minimiza peajes y tiempo
Con aprendizaje	Aprende de la experiencia	Motor de recomendación

Tareas cognitivas automatizables

La IA automatiza tareas cognitivas que antes requerían juicio humano:

- **Extracción**: leer documentos (OCR), extraer entidades (NER), capturar datos de facturas.
- **Clasificación**: spam, incidencias, prioridades, y reconocimiento de voz (habla a texto).
- **Generación**: redactar respuestas, resúmenes o informes (IA generativa).

4. Razonamiento impreciso: lógica difusa (RA2-d)

d.martinezpena@edu.gva.es

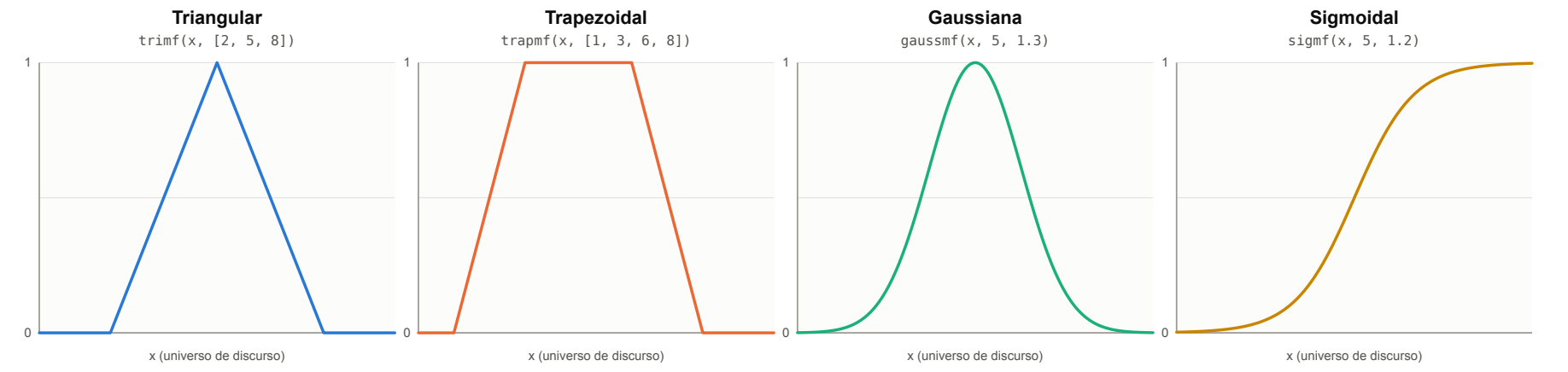
De lo booleano a lo difuso

La lógica clásica solo admite verdadero/falso. La **lógica difusa** (Zadeh, 1965) permite grados de verdad entre 0 y 1.

Modela la **vaguedad** del lenguaje humano — la probabilidad, en cambio, modela la incertidumbre.

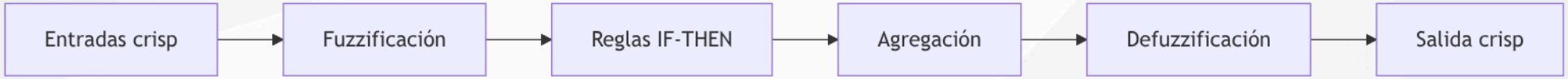
Ejemplo: "18 °C es frío con 0,7" en vez de "18 °C es frío: sí/no".

Funciones de pertenencia



Se recomiendan entre 3 y 7 curvas por variable, solapadas para que no existan huecos.

El sistema de inferencia difuso (FIS)



Dos familias: **Mamdani** (salida difusa defuzzificada, la usa `scikit-fuzzy`) y **Sugeno/TSK** (salida funcional).

Lógica difusa en el mundo real

Metro de Sendai (frenado y aceleración) · autofocus de Canon (13 reglas, 1,1 KB) · ABS de los frenos · lavadoras y aires acondicionados.

Control de tráfico: ajuste de semáforos según el flujo vehicular en tiempo real.

Apoyo al diagnóstico médico: valoración preliminar cuando los resultados de las pruebas no son concluyentes.

Práctica: el problema de la propina

```
calidad = ctrl.Antecedent(np.arange(0, 11, 1), 'calidad')
servicio = ctrl.Antecedent(np.arange(0, 11, 1), 'servicio')
propina = ctrl.Consequent(np.arange(0, 26, 1), 'propina')

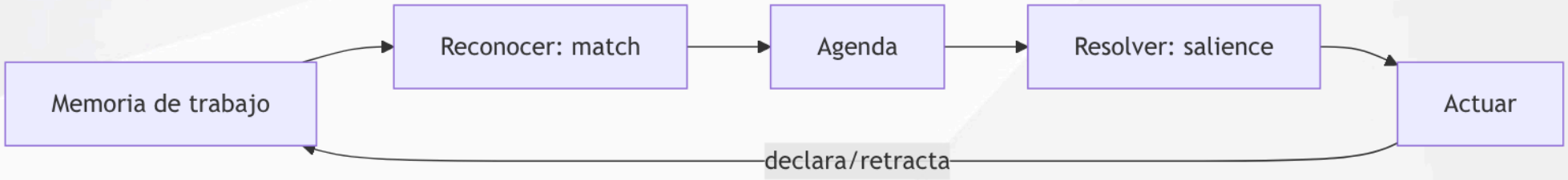
regla1 = ctrl.Rule(calidad['mala'] | servicio['pobre'], propina['baja'])
sim.compute()
# Propina sugerida: ~20%
```

`scikit-fuzzy==0.5.0`: fija la versión, es una librería semi-mantenida.

5. Sistemas basados en reglas (RA2-e)

d.martinezpena@edu.gva.es

El ciclo reconocer-actuar



Motores industriales usan el algoritmo **RETE** (Forgy, 1974) para no reevaluar todas las reglas.

Encadenamiento, CLIPS y experta

Hacia delante (data-driven): parte de los hechos, deduce hechos nuevos — lo usan CLIPS y `experta`.

Hacia atrás (goal-driven): parte de una meta y pregunta solo lo necesario — propio del diagnóstico (MYCIN).

`experta`: librería Python inspirada en CLIPS, fork de `pyknow`, motor RETE con sintaxis nativa.

Reglas en el día a día

Sistemas de recomendación: Amazon sugiere productos con reglas sobre el historial de compras.

Soporte al diagnóstico médico: reglas sobre síntomas dan una recomendación preliminar antes de la atención profesional.

Ventaja frente al ML: el razonamiento es **transparente y explicable**.

Práctica con experta

```
# PARCHE para Python 3.10+
if not hasattr(collections, 'Mapping'):
    collections.Mapping = collections.abc.Mapping
from experta import *

class DiagnosticoVehiculo(KnowledgeEngine):
    @Rule(Fact(bateria="descargada"), Fact(luces="no_encienden"))
    def bateria(self):
        self.declare(Fact(causa="bateria_descargada"))
```

Sistemas expertos que cambiaron su sector

Sistema	Origen	Dato relevante
MYCIN	Stanford, 1972	65-70 % de acierto
XCON/R1	DEC, 1978	~25 M$/año de ahorro
SID	DEC, años 80	93 % de puertas del VAX 9000

Reglas si el dominio está acotado y hace falta explicabilidad; **ML** si hay datos y patrones complejos.

6. Adecuación del modelo (RA2-f)

d.martinezpena@edu.gva.es

Cinco criterios para elegir modelo

Datos disponibles · **explicabilidad** exigida · **coste** de cómputo y mantenimiento · **precisión** requerida · **tiempo real** o latencia admisible.

La regla de oro: empieza simple. Reglas o heurística → árbol/regresión → ensamble → deep learning, solo si hace falta.

El teorema **No Free Lunch** (Wolpert y Macready, 1997): ningún algoritmo es mejor en promedio sobre todos los problemas.

Caso de estudio: Robocode

d.martinezpena@edu.gva.es

Robocode como sistema de resolución de problemas

Representación: el estado del bot y del campo de batalla en variables legibles cada turno. **Razonamiento**: reglas fijas, lógica difusa o una combinación para decidir disparo y movimiento.

Eficiencia: decisión en tiempo real, turno a turno. **Adecuación (CE f)**: sin datos previos, el punto de partida es la regla o heurística, no el ML.

Puntos clave de la unidad

Un SRP se formaliza con estado inicial, acciones, transición, objetivo y coste; BFS/DFS/A* recorren el espacio de estados con garantías distintas. Los modelos se clasifican por aprendizaje, análisis y base (conocimiento/datos).

RPA hace, la IA piensa; la difusa modela vaguedad, las reglas ejecutan reconocer-actuar. Elegir modelo (CE f): datos, explicabilidad, coste, precisión, tiempo real — empieza simple.

La unidad en la práctica

4 entregas: `N01` control difuso (`scikit-fuzzy`) · `N02` sistema de reglas (`experta`) · `T01` preparar el entorno · `T02` Robocode Tank Royale.

`T02` • **Robocode** es la de más peso, del 23 de noviembre al 3 de diciembre. `T01` es **hecho / no hecho**, como los talleres de GitHub y Markdown de la UD00.

Evaluación

Peso	Instrumento
40 %	`T02` Robocode + `N01` y `N02`
60 %	Prueba escrita del RA2

Hace falta un **5 o más** en el RA para superarlo.

¿Preguntas?

Diapositivas, apuntes y talleres en el sitio del módulo.

d.martinezpena@edu.gva.es